

# Artificial Neural Networks

## الشبكات العصبية الاصطناعية

Neurons, Activation Functions, Loss, Gradients, and Backpropagation

العصبونات ودوال التنشيط ودالة الخطأ والتدرجات والانتشار الخلفي

Prepared by | إعداد: Dr.-Ing. Aouss Gabash

### Learning Objectives

- Understand the mathematical model of an artificial neuron.
- Explain weights, bias, weighted sum, and activation functions.
- Understand the Sigmoid activation function.
- Derive the derivative of the Sigmoid function.
- Understand forward propagation.
- Define and interpret a loss function.
- Understand gradient descent.
- Use the chain rule to derive backpropagation equations.
- Apply neural networks to power-system problems.

### أهداف التعلم

- فهم النموذج الرياضي للعصبون الاصطناعي.
- فهم الأوزان والانزياح والمجموع الموزون ودوال التنشيط.
- فهم دالة Sigmoid.
- اشتقاق مشتق دالة Sigmoid خطوة بخطوة.
- فهم الانتشار الأمامي.
- تعريف دالة الخطأ وفهم معناها.
- فهم خوارزمية الانحدار المتدرج.

- استخدام قاعدة السلسلة لاشتقاق معادلات الانتشار الخلفي.
- تطبيق الشبكات العصبية على مسائل نظم الطاقة الكهربائية.

## 1. From Biological Neuron to Artificial Neuron

Artificial neural networks are inspired by the idea of biological neurons: receive signals, combine them, and generate an output.



An artificial neuron is fundamentally a mathematical function.

## 1. من العصبون الحيوي إلى العصبون الاصطناعي

تستلهم الشبكات العصبية الاصطناعية فكرتها بصورة مبسطة من العصبونات الحيوية: استقبال إشارات، دمجها، ثم إنتاج خرج.

العصبون الاصطناعي في جوهره عبارة عن دالة رياضية.

## 2. Mathematical Model of a Neuron

Suppose the neuron receives  $n$  inputs:

$$x_1, x_2, \dots, x_n$$

Each input has a corresponding weight:

$$w_1, w_2, \dots, w_n$$

The neuron first calculates the weighted sum:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

or more compactly:

$$z = \sum w_i x_i + b$$

The output is:

$$y = \varphi(z)$$

## 2. النموذج الرياضي للعصبون

لنفترض أن العصبون يستقبل  $n$  من المدخلات:

$$x_1, x_2, \dots, x_n$$

ولكل مدخل وزن:

$$w_1, w_2, \dots, w_n$$

يقوم العصبون أولاً بحساب المجموع الموزون:

$$z = \sum w_i x_i + b$$

ثم يتم تمرير  $z$  عبر دالة التنشيط:

$$y = \varphi(z)$$

### 3. Why Do We Need Weights?

The weights determine how strongly each input influences the output.

$$z = w_1 x_1 + w_2 x_2$$

If:

$$|w_1| > |w_2|$$

then input  $x_1$  has a stronger influence than  $x_2$ .

### 3. لماذا نحتاج إلى الأوزان؟

تحدد الأوزان مقدار تأثير كل مدخل على خرج العصبون.

$$z = w_1x_1 + w_2x_2$$

فإذا كان:

$$|w_1| > |w_2|$$

فإن تأثير  $x_1$  على الخرج أكبر من تأثير  $x_2$ .

## 4. What Does the Bias Do?

The bias shifts the activation function and allows the neuron to produce useful outputs even when all inputs are zero.

$$z = \sum w_i x_i + b$$

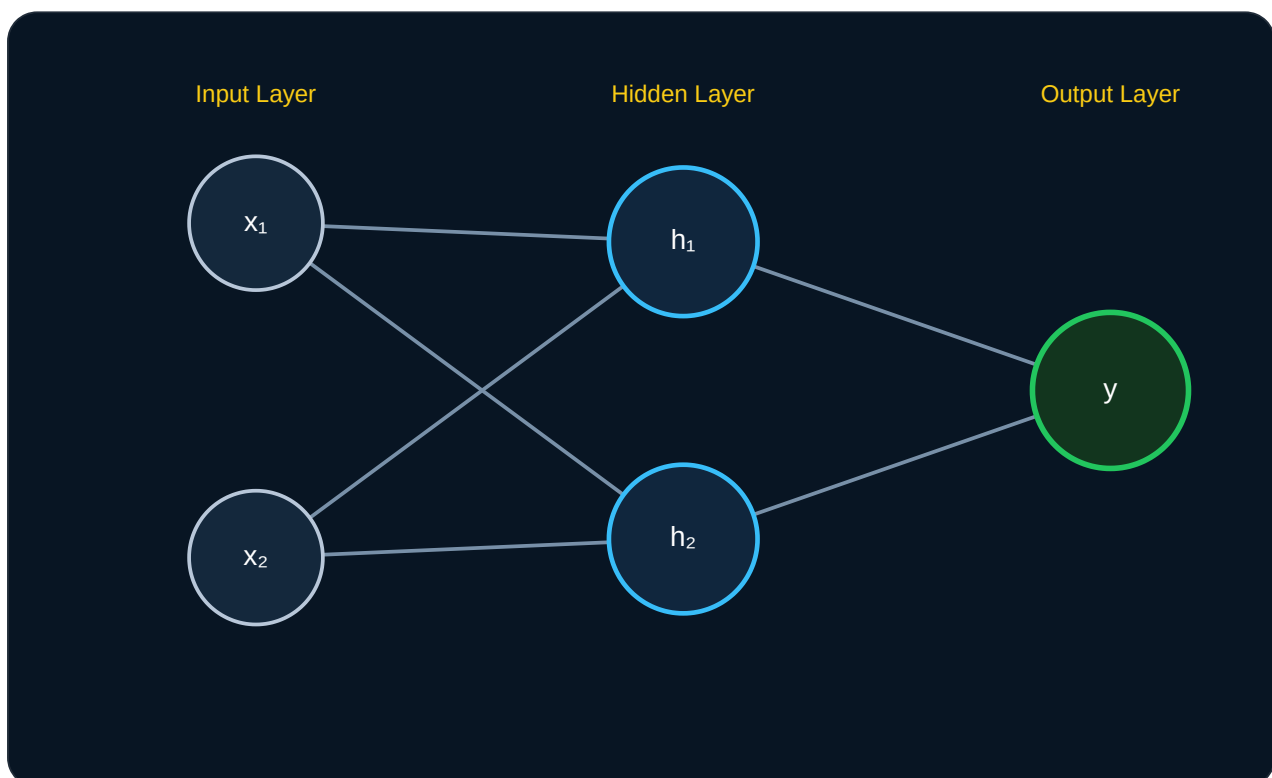
Weights control the slope and importance of inputs. Bias controls the horizontal shift of the neuron response.

#### 4. ما وظيفة الانزياح Bias؟

يسمح الانزياح بتحريك استجابة العصبون ويساعده على إعطاء خرج مناسب حتى عندما تكون بعض المدخلات مساوية للصفر.

الأوزان تحدد تأثير المدخلات، بينما يساعد الانزياح على تغيير موضع دالة التنشيط.

### 5. Structure of a Simple Neural Network



This is a 2–2–1 neural network:

2 Inputs  $\rightarrow$  2 Hidden Neurons  $\rightarrow$  1 Output

## 5. بنية شبكة عصبية بسيطة

يوضح الشكل شبكة عصبية من النوع:

$2 \rightarrow 2 \rightarrow 1$

أي مدخلان، عصبونان في الطبقة المخفية، وخرج واحد.

## 6. Activation Functions

Without an activation function, a neural network would behave mainly like a linear mathematical model.

### Sigmoid

$$\sigma(z) = 1/(1 + e^{-z})$$

### Tanh

$$\tanh(z)$$

### ReLU

$$\text{ReLU}(z) = \max(0, z)$$

## 6. دوال التنشيط

تسمح دوال التنشيط للشبكة بتمثيل علاقات غير خطية ومعقدة.

من أشهرها Sigmoid و Tanh و ReLU.

## 7. The Sigmoid Function

$$\sigma(z) = 1/(1 + e^{-z})$$

The output always lies between 0 and 1:



$$0 < \sigma(z) < 1$$

This is useful when the output can be interpreted as a probability or normalized response.

## 7. دالة Sigmoid

$$\sigma(z) = 1/(1 + e^{-z})$$

تكون قيمة الخرج دائمًا بين الصفر والواحد:

$$0 < \sigma(z) < 1$$

## 8. Derivative of the Sigmoid Function

Start with:

$$\sigma(z) = (1 + e^{-z})^{-1}$$

Differentiate using the chain rule:

$$d\sigma/dz = -(1 + e^{-z})^{-2} \times (-e^{-z})$$

Therefore:

$$d\sigma/dz = e^{-z}/(1 + e^{-z})^2$$

Now note:

$$\sigma(z) = 1/(1 + e^{-z})$$

and:

$$1 - \sigma(z) = e^{-z}/(1 + e^{-z})$$

Multiplying both expressions:

$$\sigma(z)[1 - \sigma(z)] = e^{-z}/(1 + e^{-z})^2$$

Hence:

$$\sigma'(z) = \sigma(z)[1 - \sigma(z)]$$

This compact derivative is one reason Sigmoid is convenient for learning derivations.

## 8. اشتقاق دالة Sigmoid

نبدأ من:

$$\sigma(z) = (1 + e^{-z})^{-1}$$

باستخدام قاعدة السلسلة نحصل على:

$$\sigma'(z) = e^{-z} / (1 + e^{-z})^2$$

وبإعادة ترتيبها باستخدام قيمة  $\sigma$  نفسها:

$$\sigma'(z) = \sigma(z)[1 - \sigma(z)]$$

أي أننا لا نحتاج في التطبيق إلى إعادة حساب المشتقة من البداية؛ يمكن حسابها مباشرة باستخدام قيمة خرج Sigmoid.

## 9. Forward Propagation

Forward propagation means calculating the network output from the input toward the output layer.

### Hidden neuron 1

$$z_1 = w_1x_1 + w_2x_2 + b_1$$

$$h_1 = \sigma(z_1)$$

### Hidden neuron 2

$$z_2 = w_3x_1 + w_4x_2 + b_2$$

$$h_2 = \sigma(z_2)$$

### Output neuron

$$z_3 = v_1h_1 + v_2h_2 + b_3$$

$$y = \sigma(z_3)$$

## 9. الانتشار الأمامي

يعني الانتشار الأمامي حساب خرج الشبكة بدءًا من المدخلات، مرورًا بالطبقات المخفية، وصولًا إلى الخرج النهائي.

Input → Hidden Layer → Output

## 10. Loss Function

The network prediction  $y$  must be compared with the desired target  $t$ .

A simple squared-error loss is:

$$E = 1/2(y - t)^2$$

The factor  $1/2$  is used because it simplifies the derivative.

## 10. دالة الخطأ

نقارن خرج الشبكة  $y$  مع القيمة المطلوبة  $t$ .

$$E = 1/2(y - t)^2$$

نستخدم العامل 1/2 لأنه يجعل الاشتقاق أبسط.

## 11. Derivative of the Loss

Start from:

$$E = 1/2(y - t)^2$$

Differentiate with respect to y:

$$\partial E / \partial y = y - t$$

If y is larger than t, the derivative is positive. If y is smaller than t, the derivative is negative.

## 11. مشتقة دالة الخطأ

نبدأ من:

$$E = 1/2(y - t)^2$$

وبالاشتقاق بالنسبة إلى  $y$ :

$$\partial E / \partial y = y - t$$

هذه القيمة تخبرنا باتجاه الخطأ.

## 12. Why Do We Need Backpropagation?

We know the final error, but we need to know:

How much did each weight contribute to the error?

Backpropagation answers this question using derivatives and the chain rule.

## 12. لماذا نحتاج إلى الانتشار الخلفي؟

نعرف الخطأ النهائي، لكننا نحتاج إلى معرفة مقدار مساهمة كل وزن في هذا الخطأ.

الانتشار الخلفي يستخدم المشتقات وقاعدة السلسلة لحساب تأثير كل وزن.

## 13. The Chain Rule

If:

$$E \rightarrow y \rightarrow z \rightarrow w$$

then:

$$\partial E / \partial w = \partial E / \partial y \times \partial y / \partial z \times \partial z / \partial w$$

Backpropagation is essentially repeated application of the chain rule.

### 13. قاعدة السلسلة

إذا كان الوزن  $w$  يؤثر على  $z$ ، و  $z$  يؤثر على  $y$ ، ولا يؤثر على الخطأ  $E$ ، فإن تأثير  $w$  على  $E$  يحسب بضرب المشتقات الجزئية على طول المسار.

$$\partial E / \partial w = \partial E / \partial y \times \partial y / \partial z \times \partial z / \partial w$$

## 14. Output-Layer Error Signal

For the output neuron:



$$y = \sigma(z_3)$$

We already know:

$$\partial E / \partial y = y - t$$

and:

$$\partial y / \partial z_3 = y(1 - y)$$

Therefore:

$$\delta_3 = \partial E / \partial z_3 = (y - t)y(1 - y)$$

This is the origin of the MATLAB expression:

$$dO = (y - t) * y * (1 - y)$$

## 14. إشارة الخطأ في طبقة الخرج

لدينا:

$$y = \sigma(z_3)$$

وباستخدام قاعدة السلسلة:

$$\delta_3 = (y - t)y(1 - y)$$

ولهذا تظهر في كود MATLAB العلاقة:

$$dO = (y - t) * y * (1 - y)$$

## 15. Gradient of an Output Weight

For weight  $v_1$ :

$$z_3 = v_1 h_1 + v_2 h_2 + b_3$$

Therefore:

$$\partial z_3 / \partial v_1 = h_1$$

Thus:

$$\partial E / \partial v_1 = \delta_3 h_1$$

Similarly:

$$\partial E / \partial v_2 = \delta_3 h_2$$

and for the bias:

$$\partial E / \partial b_3 = \delta_3$$

## 15. تدرج وزن في طبقة الخرج

لأن:

$$z_3 = v_1 h_1 + v_2 h_2 + b_3$$

فإن:

$$\partial z_3 / \partial v_1 = h_1$$

$$\partial E / \partial v_1 = \delta_3 h_1$$

## 16. Hidden-Layer Error Signal

Hidden neuron  $h_1$  influences the output through weight  $v_1$ .

Using the chain rule:

$$\delta_1 = \delta_3 v_1 h_1 (1 - h_1)$$

Similarly:

$$\delta_2 = \delta_3 v_2 h_2 (1 - h_2)$$

The output error is propagated backward through the connection weights.

## 16. إشارة الخطأ في الطبقة المخفية

يصل تأثير الخطأ من طبقة الخرج إلى الطبقة المخفية عبر الأوزان.

$$\delta_1 = \delta_3 v_1 h_1 (1 - h_1)$$

$$\delta_2 = \delta_3 v_2 h_2 (1 - h_2)$$

## 17. Gradient of Input-to-Hidden Weights

If:

$$z_1 = w_1 x + b_1$$

then:

$$\partial z_1 / \partial w_1 = x$$

Therefore:

$$\partial E / \partial w_1 = \delta_1 x$$

and:

$$\partial E / \partial b_1 = \delta_1$$

## 17. تدرجات الأوزان بين الدخل والطبقة المخفية

لأن:

$$z_1 = w_1 x + b_1$$

فإن:

$$\partial E / \partial w_1 = \delta_1 x$$

$$\partial E / \partial b_1 = \delta_1$$

## 18. Gradient Descent

After calculating the gradients, we update every parameter:

$$w(new) = w(old) - \eta \partial E / \partial w$$

where  $\eta$  is the learning rate.

We move in the opposite direction of the gradient because the gradient points toward increasing error.

## 18. الانحدار المتدرج

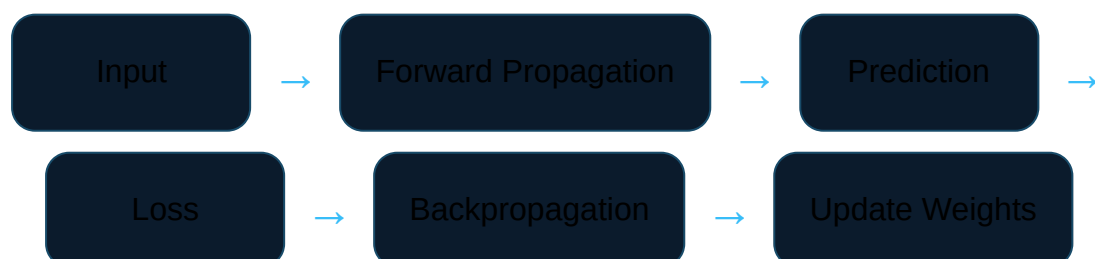
بعد حساب المشتقات نقوم بتحديث الأوزان:

$$w(new) = w(old) - \eta \partial E / \partial w$$

حيث  $\eta$  هو معدل التعلم.

نطرح التدرج لأننا نريد التحرك باتجاه انخفاض الخطأ وليس زيادته.

## 19. Complete Training Cycle



Repeat until the error becomes sufficiently small.

## 19. دورة التدريب الكاملة

المدخلات ← الانتشار الأمامي ← التنبؤ ← الخطأ ← الانتشار الخلفي ← تحديث الأوزان ← التكرار

## 20. Numerical Example

Assume:

$$x = 1$$

$$w = 0.5$$

$$b = 0$$

Then:



$$z = wx + b = 0.5$$

Sigmoid output:

$$y = \sigma(0.5) \approx 0.622$$

Suppose the target is:

$$t = 1$$

Loss:

$$E = 1/2(0.622 - 1)^2$$

$$E \approx 0.0714$$

**20. مثال عددي**

إذا كان:

$$x = 1, w = 0.5, b = 0$$

فإن:

$$z = 0.5$$

$$y \approx 0.622$$

وإذا كانت القيمة المطلوبة  $t=1$  فإن الخطأ:

$$E \approx 0.0714$$

## 21. MATLAB Connection

Forward propagation:

$$z1 = W.w1 * x + W.b1$$

$$h1 = \sigma(z1)$$

Output-layer error:

$$dO = (y - t) * y * (1 - y)$$

Output-weight gradients:

$$dV1 = dO * h1$$

$$dV2 = dO * h2$$

Hidden-layer error:

$$d1 = dO * W.v1 * h1 * (1 - h1)$$

Weight gradient:

$$dW1 = d1 * x$$

## 21. الربط مع MATLAB

الآن يمكن فهم كل سطر في كود الانتشار الخلفي بصورة رياضية.

كل قيمة  $d$  تمثل مشتقة جزئية أو إشارة خطأ مستخدمة لتحديد اتجاه تحديث الوزن.

## 22. Power-System Application: Load Forecasting

A neural network can receive several power-system variables:

$$x = [Temperature, Hour, PreviousLoad, DayType]$$

and predict:

$$y = FutureElectricalLoad$$



## 22. تطبيق في نظم الطاقة: التنبؤ بالحمل

يمكن للشبكة العصبية استقبال معلومات مثل درجة الحرارة والوقت والحمل السابق ونوع اليوم.

ثم تقوم بإنتاج تنبؤ للحمل الكهربائي المستقبلي.

## 23. Other Power-System Applications

### PV Forecasting

Predict photovoltaic power generation.

### Wind Forecasting

Predict wind-power generation.

### Fault Classification

Identify and classify network faults.

### Voltage Estimation

Estimate voltage states using measurements.

### Battery Management

Estimate SOC or predict optimal operation.

### Smart Grid Monitoring

Detect abnormal conditions from large datasets.

## 23. تطبيقات أخرى في نظم الطاقة

- التنبؤ بالطاقة الشمسية.
- التنبؤ بطاقة الرياح.
- تصنيف الأعطال.
- تقدير الجهد والحالة التشغيلية.
- إدارة أنظمة تخزين الطاقة.
- مراقبة الشبكات الذكية.

## 24. Summary

- A neuron calculates a weighted sum and applies an activation function.
- The fundamental neuron equation is  $z = \sum w_i x_i + b$ .
- Sigmoid maps values into the interval (0,1).
- Its derivative is  $\sigma'(z) = \sigma(z)[1 - \sigma(z)]$ .
- Forward propagation calculates network predictions.
- The loss function measures prediction error.
- Backpropagation calculates gradients using the chain rule.
- Gradient descent updates the weights to reduce error.
- Neural networks have many applications in modern power systems.

## 24. الخلاصة

- يحسب العصبون مجموعاً موزوناً ثم يطبق دالة تنشيط.
- المعادلة الأساسية للعصبون هي  $z = \sum w_i x_i + b$ .
- تحول Sigmoid القيم إلى المجال بين الصفر والواحد.
- مشتقتها هي  $\sigma'(z) = \sigma(z)[1 - \sigma(z)]$ .
- يحسب الانتشار الأمامي خرج الشبكة.
- تقيس دالة الخطأ الفرق بين التنبؤ والقيمة المطلوبة.
- يحسب الانتشار الخلفي التدرجات باستخدام قاعدة السلسلة.
- يحدث الانحدار المتدرج الأوزان لتقليل الخطأ.
- للشبكات العصبية تطبيقات واسعة في نظم الطاقة الكهربائية.



## Review Questions

1. What is the mathematical equation of an artificial neuron?
2. What is the role of weights?
3. What is the role of the bias?
4. Why are activation functions required?
5. Derive the derivative of the Sigmoid function.
6. What is forward propagation?
7. Why do we need a loss function?
8. Explain the chain rule in backpropagation.
9. Derive  $dO = (y-t)y(1-y)$ .
10. Why do we subtract the gradient during training?
11. Name four neural-network applications in electrical power systems.

## أسئلة المراجعة

1. ما المعادلة الرياضية للعصبون الاصطناعي؟
2. ما وظيفة الأوزان؟
3. ما وظيفة الانزاح؟
4. لماذا نحتاج إلى دوال التنشيط؟
5. اشتق مشتقة دالة Sigmoid.
6. ما المقصود بالانتشار الأمامي؟
7. لماذا نحتاج إلى دالة خطأ؟
8. اشرح قاعدة السلسلة في الانتشار الخلفي.
9. اشتق العلاقة  $dO = (y-t)y(1-y)$ .
10. لماذا نطرح التدرج أثناء التدريب؟
11. اذكر أربعة تطبيقات للشبكات العصبية في نظم الطاقة الكهربائية.